

Payment Request Module

Technical Documentation & Code Review

Malabar Group SARL - Lubumbashi, DRC

Document	Payment Request Module - Technical Documentation & Code Review
System	Malabar Group ERP - custom PHP MVC (Database::customQuery / viewWithLayout pattern)
Components	app/controllers/PaymentController.php, app/views/payment/payment.php, assets/pages/css/payment_request_styles.css
Stack	PHP 7.4+ / MySQL / Bootstrap 5 / jQuery / DataTables (server-side) / SheetJS / Toastr
Scope	Multi-location, multi-stage payment request approval workflow with MCA reference tracking, document uploads, Excel import/export and printable "Demande de Fonds" output
Audience	Developers and maintainers of the Malabar ERP

Contents

1. Module Overview
2. Architecture & Request Flow
3. Data Model
4. Roles & Permission Matrix
5. Approval Workflow (State Machine)
6. Record Visibility Model
7. Controller API Reference
8. MCA Reference Lifecycle
9. File Upload & Document Handling
10. Frontend Reference (payment.php)
11. Security Analysis
12. Performance Analysis
13. Code Quality Findings
14. Recommended Fixes (paste-ready)
15. Suggested Database Indexes
16. Appendix - Endpoint Index

1. Module Overview

The Payment Request module ("Demande de Fonds") is the internal cash-and-bank disbursement workflow of the Malabar ERP. A staff member raises a request for funds, attaches supporting documents, links it to one or more MCA tracking references, and the request then travels through a fixed chain of approvals before being marked as paid. The module is location-aware: Lubumbashi (default), Kolwezi (location 2) and location 3 each have their own approval roles, and a request is only actionable by roles bound to its own location.

Functional capabilities

- **Request capture** - department, location, beneficiary, requestee, client, payment purpose, currency, amount, payment type (Cash/Bank), expense type, motif and two supporting documents.
- **MCA reference linking** - a request can be split across up to 50 tracking references, each with its own amount; the sum of reference amounts must equal the header amount.
- **Reference validation** - each reference is checked for existence in the relevant tracking table (`imports_t`, `exports_t`, `locals_t`) and for duplicate usage against the same expense type.
- **Five-stage approval** - Department, Finance, Management, Under Process (Bank only) and Paid, each with an approve or reject action, an actor, a timestamp and a note field.
- **Rejection & resubmission** - a rejected request can be edited by its creator; saving it clears every approval field so the chain restarts from the beginning.
- **Dashboard** - eight clickable status cards drive a server-side DataTable filter.
- **Output** - browser print layout that reproduces the two-page official "Demande de Fonds" form, plus an Excel export and a mailto-based notification for expense type 42.

Key design decisions

Decision	Implementation	Consequence
Approval state as tri-state integers	Each stage column holds NULL (pending), 1 (approved) or -1 (rejected)	Simple SQL filters; no separate status column to keep in sync
MCA references stored as JSON	<code>mca_data</code> holds a JSON array of <code>{mca_ref, amount}</code>	No child table needed, but references are not indexable or joinable
Role IDs hardcoded as class constants	17 constants plus role-to-action arrays inside the controller	Explicit and readable, but the same matrix is duplicated in JavaScript
Server-side DataTables	<code>get_list()</code> builds WHERE, count, total and page queries	Scales to large tables; three queries per draw
Visibility centralised	<code>buildVisibilityConditions()</code> returns conditions + bound params	One place to change who sees what; used by list, counts and export

2. Architecture & Request Flow

The module follows the established ERP conventions: a single controller extending `Controller`, all persistence through `Database::customQuery()` / `selectData()` / `insertData()` / `updateData()`, and one view rendered through `viewWithLayout()`. Everything after the initial page render is AJAX; the view never reloads.

```

Browser (payment.php)
|
|-- GET payment/                -> index()                render page + dropdowns
|-- GET payment/get_list        -> get_list()            DataTables server-side
|-- POST payment/get_status_counts -> get_status_counts()  8 card counters
|-- POST payment/store          -> store()                insert | update (multipart)
|-- POST payment/get_single     -> get_single()           view / edit / print / email
|-- POST payment/getExpenseTypesByCategory -> filter expense types by pay_for
|-- POST payment/get_mca_refs_by_client -> reference picker (+ perdiem when expense_type = 28)
|-- POST payment/validate_mca_exists -> reference exists in tracking table

```

```

|-- POST payment/check_mca_duplicate      -> reference not already consumed
|-- POST payment/update_approval          -> dept | finance | management | paid
|-- POST payment/under_process_approval  -> Bank-only intermediate stage (approve | reject)
|-- POST payment/reject_request           -> rejection at any of the four main stages
|-- POST payment/export_to_excel          -> rows for client-side XLSX build
|-- GET payment/delete/{id}               -> delete() (management only)

```

Cross-cutting helpers

Helper	Responsibility	Notes
__construct()	Starts session, seeds CSRF token, creates and permission-checks the upload directory	Writes 2-4 error_log lines on every request, including successes
validateCSRF()	Constant-time comparison of \$_POST[csrf_token]	Only enforced when REQUEST_METHOD is POST - GET endpoints are unprotected
validateSession()	Requires user_id and user_data.role_id	Returns an array; not called by get_single() or check_mca_duplicate()
jsonResponse()	Emits {success, message, data} and exits	Consistent envelope consumed by the JS ajaxRequest() wrapper
isFullViewRole()	Roles 1, 3, 4, 47, 48 bypass all visibility filters	Used inside buildVisibilityConditions()
buildVisibilityConditions()	Builds row-level filter for the current role, with an optional table alias	Shared by get_list(), get_status_counts() and export_to_excel()
validateApprovalWorkflow()	Enforces stage ordering before an approval is written	Does not cover the Under Process stage - see finding S-04

3. Data Model

The columns below are inferred from every read and write performed by the controller and the view.

3.1 payment_requests

Column	Type (inferred)	Purpose
id	INT PK AUTO_INCREMENT	Request number, printed on the form as "DEMANDE DE FONDS No."
department	INT FK -> department_master_t.id	Requesting department
location_id	INT FK -> main_office_master_t.id	Drives which location roles may act (1, 2, 3, other)
client_id	INT NULL FK -> clients_t.id	Nullable; blanked when non-numeric
beneficiary	VARCHAR(200)	Who receives the funds
requestee	VARCHAR(200)	Session full name, read-only in the form
pay_for	TINYINT '0'..'4'	0 Import, 1 Export, 2 Local, 3 Other, 4 Pre Payment
currency	INT FK -> currency_master_t.id	Header currency
amount	DECIMAL(15,2)	Header amount; must equal the sum of MCA amounts
payment_type	ENUM('Cash','Bank')	Bank adds the Under Process stage
expense_type	INT FK -> expense_type_master_t.id	Filtered by pay_for; 25 and 28 and 42 have special behaviour
motif	VARCHAR(500)	Reason, 10-500 chars enforced client-side only
cash_collector	VARCHAR(100) NULL	Captured at the Paid stage (mandatory there)
chargeback	DECIMAL(15,2) NULL	Optional, captured at Department approval
mca_ref	VARCHAR NULL	First reference only - denormalised convenience copy
mca_data	JSON / TEXT	Array of {"mca_ref": "...", "amount": 0.00}
file1_path .. file4_path	VARCHAR NULL	1-2 request documents, 3-4 payment proof documents
dept_approval	TINYINT NULL	NULL pending / 1 approved / -1 rejected
dept_approved_at / _by / dept_notes	DATETIME / INT / TEXT	Actor, time and rejection reason
finance_approval + 3 columns	same shape	Finance stage
management_approval + 3 columns	same shape	Management stage
under_process + 3 columns	same shape	Bank-only stage (under_process_notes)
paid_approval + 3 columns	same shape	Final stage (paid_notes)
created_at / created_by	DATETIME / INT	Ownership - drives the edit button and visibility
updated_at / updated_by	DATETIME / INT NULL	Set on update only

3.2 Related tables

Table	Used for
department_master_t	Department dropdown and list join
main_office_master_t	Location dropdown; first two letters seed OTH-/PRE- auto references
clients_t	Client dropdown (short_name displayed)

Table	Used for
currency_master_t	Currency dropdown, filtered by the second argument of selectData()
expense_type_master_t	Expense types with per-category flags: import, export, local, other, advance
users_t	Approver names resolved in get_single() via four LEFT JOINS
imports_t / exports_t	Reference validation on mca_ref (plain or JSON), import rows exclude clearing_status = 7
locals_t	Reference validation on mca_lt_reference
perdiem_master_t	Per-diem amount pre-fill when expense_type = 28 (client + goods type + transport mode)

4. Roles & Permission Matrix

Seventeen role constants are declared. The table below merges the constants with the `$rolePermissions` maps in `update_approval()` and `reject_request()`, the guard in `under_process_approval()`, and the button-rendering logic in the view.

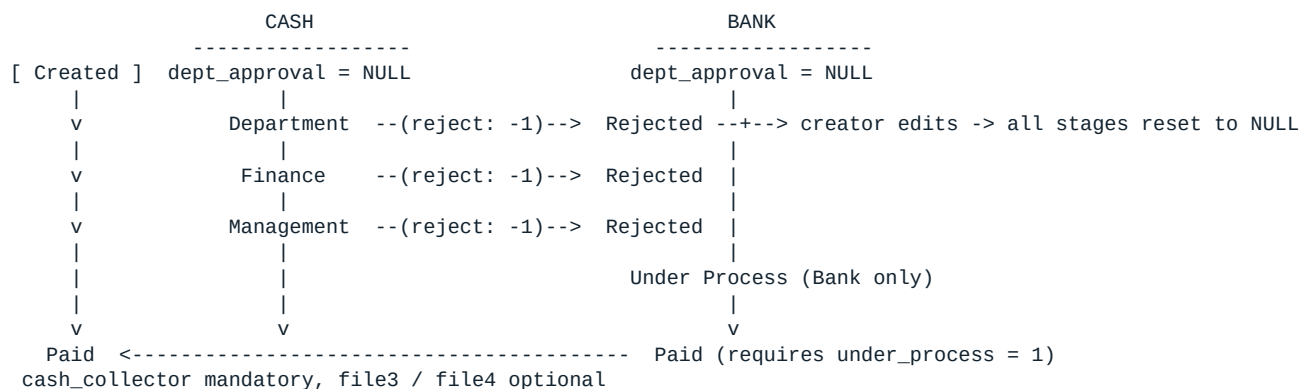
ID	Constant	Location scope	Dept	Fin	Mgmt	U/P	Paid
1	ROLE_MANAGEMENT	All (full view)			Y		
3	ROLE_DEPARTMENT	All except 2 and 3 (full view)	Y				
4	ROLE_FINANCE_CASH	All except 2 (full view)		Y			
5	ROLE_FINANCE_BANK	Bank rows or own rows		Y			
10	ROLE_CASHIER	Cash outside 2 and 3, own rows					Y
11	ROLE_BANK_OFFICER	Bank rows, own rows				Y	Y
27	ROLE_LOCATION_MANAGER	Location 3 only	Y	Y	Y		Y
33	ROLE_LOCATION_2_DEPT_MGMT	Location 2 only	Y				
34	ROLE_LOCATION_2_FINANCE_PAID	Location 2 only		Y			Cash only
39	ROLE_LOCATION_1_MANAGER	Location 1 only			Y		
47	ROLE_CASH_BANK_OFFICER	All (full view)		Y			
48	ROLE_BANK_OFFICER_FULL_VIEW	All (full view)				Y	Y
53	ROLE_LOCATION_3_CASHIER	Location 3 only					Y
54	ROLE_LOCATION_2_MGMT	Location 2 only			Y		

Note. Role 27 (Location 3 Manager) can perform *all four* stages on its own location, which means a single user can approve a request end to end with no segregation of duties. Role 34 is granted `paid_approve` in the permission map but is then blocked at runtime for Bank payments by an explicit guard in `handleApproval()` and `reject_request()`.

4.1 Duplicated matrix in the view

The same role numbers are hardcoded a second time in the DataTable action column (`roleId == 33, == 27, == 47` and so on). The two copies have already drifted: the comment above the Under Process branch says "Role 11, 48, 34" but the condition only tests 11 and 48, and the server only allows 11 and 48. See finding Q-01.

5. Approval Workflow (State Machine)



5.1 Stage guards enforced server-side

Stage	Precondition checked	Enforced in
Department	None beyond role and location; cannot re-approve an already approved stage	update_approval() + validateApprovalWorkflow()
Finance	dept_approval = 1	validateApprovalWorkflow("finance_approve")
Management	dept_approval = 1 AND finance_approval = 1	validateApprovalWorkflow("management_approve")
Under Process	payment_type = Bank, all three prior stages = 1, not already 1 or -1	under_process_approval() (its own inline checks)
Paid	dept, finance and management all = 1; cash_collector non-empty	validateApprovalWorkflow("paid_approve") + handleApproval()

Gap. The Paid guard never checks `under_process = 1` for Bank payments. The UI hides the Paid button until Under Process is done, but a direct POST by role 11 or 48 can skip the stage entirely. This is finding S-04.

5.2 Card filters and their SQL

Eight cards are rendered; `get_status_counts()` and `get_list()` must agree on the definitions, and they do - the predicates are duplicated verbatim between the two methods with only the `pr .` prefix differing.

Card	Definition
TOTAL	Every visible row
PENDING DEPT	dept_approval IS NULL and no later stage rejected
PENDING FINANCE	dept = 1, finance IS NULL, no later stage rejected
PENDING MGMT	dept = 1, finance = 1, management IS NULL, no later stage rejected
PENDING UNDER PROCESS	Bank, dept/finance/management = 1, under_process IS NULL, paid not rejected
PENDING PAYMENT	(Cash and three stages = 1) or (Bank and three stages = 1 and under_process = 1), paid IS NULL
PAID	paid_approval = 1
REJECTED	any of the five stage columns = -1

6. Record Visibility Model

`buildVisibilityConditions($userRole, $userId, $prefix)` returns an array of SQL fragments plus the bound parameters. The `prefix` argument lets the same logic serve both the aliased list query (`pr .`) and the unaliased counter query.

Role	Rows returned
1, 3, 4, 47, 48	All rows - no WHERE fragment is added at all
5 (Finance Bank)	<code>payment_type = Bank OR created_by = me</code>
27, 53	<code>location_id = 3 OR created_by = me</code>
33, 34, 54	<code>location_id = 2 OR created_by = me</code>
39	<code>location_id = 1 OR created_by = me</code>
10 (Cashier)	<code>created_by = me OR (Cash AND location NOT IN (2,3)) OR paid_approved_by = me</code>
11 (Bank Officer)	<code>created_by = me OR payment_type = Bank OR paid_approved_by = me</code>
Any other role	<code>created_by = me only</code>

Important. Visibility is applied to the *list*, the *counters* and the *export*, but not to `get_single()`. Any authenticated user who knows or guesses an id can read the full record, including approver names, notes and document paths. This is finding S-02.

7. Controller API Reference

`index()`

Endpoint	GET <code>payment/</code>
Purpose	Loads the six master datasets and renders the page through <code>viewWithLayout()</code> .
Input	none
Output	HTML - <code>dept, loc, client, currency, expense, csrf_token, current_user_id</code>
Notes	Currency is fetched with a second <code>selectData()</code> argument of 2, presumably an active/type flag. No role check - the page itself is reachable by any logged-in user.

`get_status_counts()`

Endpoint	POST <code>payment/get_status_counts</code>
Purpose	Returns the eight dashboard counters in a single aggregate query.
Input	<code>csrf_token</code>
Output	<code>{total, waiting_dept, waiting_finance, waiting_mgmt, waiting_under_process, waiting_payment, paid, rejected}</code>
Notes	One SELECT with eight conditional SUMs over the visible rows. Called on page load and again on every DataTable draw via <code>drawCallback</code> , so paging or sorting triggers a full re-count.

get_list()

Endpoint	GET payment/get_list
Purpose	Server-side DataTables feed: visibility filter, card filter, global search, sorting and paging.
Input	draw, start, length, status_filter, search[value], order[0][column], order[0][dir], csrf_token
Output	{draw, recordsTotal, recordsFiltered, data[]}
Notes	Search spans 11 columns including mca_data. Sorting is whitelisted through \$columnMap and LIMIT uses sprintf("%d"), so both are injection-safe. Three queries run per draw (filtered count, unfiltered count, page). Because the request is a GET, validateCSRF() is a no-op here.

store()

Endpoint	POST payment/store (multipart)
Purpose	Creates or updates a request, uploads documents and persists the MCA JSON.
Input	action, payment_id, department, location, beneficiary, requestee, client_id, pay_for, currency, amount, payment_type, expense_type, motif, cash_collector, mca_reference[], mca_amount[], file1, file2, csrf_token
Output	{success, message, data:{payment_id}}
Notes	Wrapped in beginTransaction/commit. Validates that the MCA total matches the amount within 0.01. On update, if any stage is -1 all nineteen approval fields are reset to NULL so the chain restarts. No ownership check on payment_id and no re-check that the record is still editable (finding S-01).

get_single()

Endpoint	POST payment/get_single
Purpose	Full record plus resolved approver names, absolute file URLs and decoded MCA array.
Input	id, csrf_token
Output	{data:{...record, file1_url..file4_url}, mca_refs:[...]}
Notes	Four LEFT JOINS on users_t resolve the approver names. This single endpoint backs the view modal, the edit form, all four approval modals, the print layout and the email builder. No session or visibility check.

update_approval()

Endpoint	POST payment/update_approval
Purpose	Single entry point for dept_approve, finance_approve, management_approve and paid_approve.
Input	id, action, csrf_token (+ chargeback_needed / chargeback_value for dept; cash_collector, file3, file4 for paid)
Output	{success, message}
Notes	Loads the payment, applies the location guard, checks \$rolePermissions, then delegates to handleApproval(), which re-runs validateCSRF(), validateSession() and reloads the same row - two identical SELECTs per call.

under_process_approval()

Endpoint	POST payment/under_process_approval
Purpose	Bank-only intermediate stage; handles both approve and reject.
Input	id, action (approve reject), reason (when rejecting), csrf_token
Output	{success, message}
Notes	Restricted to roles 11 and 48. Verifies payment_type = Bank and that all three prior stages are approved. Unlike the other stages it carries no location guard, so a bank officer can act on any location.

reject_request()

Endpoint	POST payment/reject_request
Purpose	Writes -1, the actor, the timestamp and the reason for one of the four main stages.
Input	id, reject_type (dept finance management paid), reason, csrf_token
Output	{success, message}
Notes	Mirrors update_approval() location and role checks with a second, near-identical copy of the matrix. It does not verify that the stage being rejected is the current stage, so a role could reject a stage that was already approved earlier in the chain.

validate_mca_exists()

Endpoint	POST payment/validate_mca_exists
Purpose	Confirms a reference exists in the tracking table for the chosen pay_for and belongs to the client.
Input	mca_ref, pay_for, client_id, csrf_token
Output	{exists, tracking_id, table} or {exists:false, wrong_client}
Notes	imports_t and exports_t are matched either directly or as a JSON substring ("%mca_ref":"..."). pay_for 3 and 4 short-circuit with skip_validation.

check_mca_duplicate()

Endpoint	POST payment/check_mca_duplicate
Purpose	Rejects a reference already consumed by another request with the same expense type.
Input	mca_ref, expense_type, payment_id, csrf_token
Output	{exists, payment_id}
Notes	Expense type 25 is explicitly exempt. Implementation loads every row for that expense type and json_decodes each one in PHP - see finding P-01.

get_mca_refs_by_client()

Endpoint	POST payment/get_mca_refs_by_client
Purpose	Populates the reference picker modal for a client.
Input	client_id, paymentfor, expenseType, location, csrf_token
Output	array of {payment_id, mca_ref, type_of_goods, transport_mode, perdiem_amount}
Notes	When expenseType = 28 it joins perdiem_master_t so the picker can pre-fill the amount. Imports exclude clearing_status = 7 (cancelled). locals_t has no goods/transport columns so those are selected as NULL.

getExpenseTypesByCategory()

Endpoint	POST payment/getExpenseTypesByCategory
Purpose	Returns expense types flagged for the selected pay_for category.
Input	pay_for, csrf_token
Output	raw JSON array of expense_type_master_t rows
Notes	Maps 0-4 to the columns import, export, local, other, advance. Echoes the array directly rather than using the jsonResponse envelope - the only endpoint that does.

export_to_excel()

Endpoint	POST payment/export_to_excel
Purpose	Returns all visible rows so the browser can build the workbook with SheetJS.
Input	csrf_token
Output	array of joined rows
Notes	No paging and no column projection - selects pr.* for the entire visible set in one response.

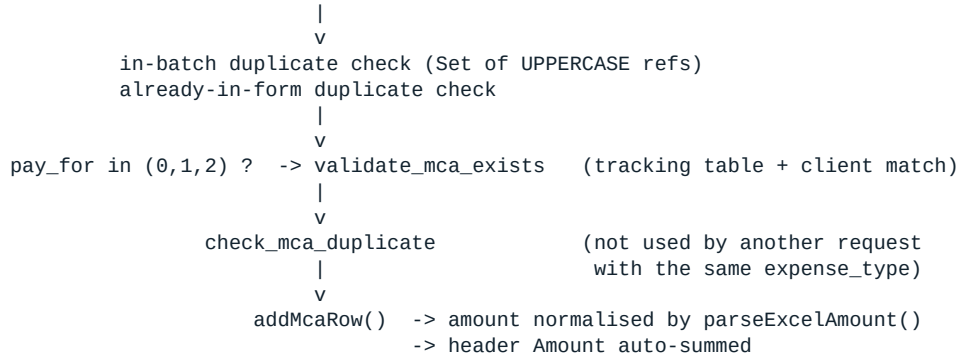
delete(\$id)

Endpoint	GET payment/delete/{id}
Purpose	Hard-deletes a request. Management role only.
Input	id in the URL
Output	{success, message}
Notes	Uses deleteData(), so it bypasses the display = "Y" soft-delete convention used elsewhere in the ERP. Because it is a GET the CSRF check does not apply. No UI currently calls it.

8. MCA Reference Lifecycle

References can enter a request through three paths, and all three converge on the same two-step validation.

- (a) Manual add N rows -> type reference -> blur event
- (b) Picker Select button -> get_mca_refs_by_client -> checkboxes -> Apply
- (c) Excel import SheetJS parse -> column 0 = reference, column 1 = amount



8.1 Special cases

Case	Behaviour
pay_for = 3 (Other)	Reference auto-generated as OTH-XX-n from the first two letters of the location name; the input is rendered read-only
pay_for = 4 (Pre Payment)	Same, with the PRE- prefix
expense_type = 25	Duplicate references are explicitly permitted
expense_type = 28	Picker joins perdiem_master_t and pre-fills the per-diem amount per reference
expense_type = 42	After a successful save the browser opens a mailto to three hardcoded recipients (Warehouse Storage Fee notification)
client_id = 68	Passed as an empty client to validate_mca_exists, effectively skipping the client-ownership check
imports_t	Rows with clearing_status = 7 (cancelled) are excluded from the picker

8.2 Amount parsing

parseExcelAmount() in the view normalises the locale variations that appear in supplier spreadsheets. It inspects the position of the last comma relative to the last dot to decide which character is the decimal separator, so "1,259.20", "1.259,20", "1 259,20" and a genuine numeric cell all yield 1259.20. The Excel reader is also configured with raw: true so numeric cells never arrive as formatted strings. Without this, parseFloat("1, 259.20") would silently yield 1.

9. File Upload & Document Handling

Control	Implementation	Assessment
Size limit	5 MB constant, plus a zero-byte rejection	Adequate
Extension whitelist	pdf, jpg, jpeg, png, doc, docx	Adequate
MIME whitelist	finfo_file() on the temp file, five accepted types	Good - checks content, not just the name
Filename	Non-alphanumerics replaced, prefixed with time() and uniqid()	Prevents traversal and collisions
Storage	UPLOAD_PATH/payments/, returned path "uploads/payments/..."	Inside the web root - see finding S-05

Control	Implementation	Assessment
Replacement	deleteOldFiles() unlinks the previous file before the new one is written	If the subsequent write fails the old document is already gone
Orphans	Files are moved before the DB transaction commits	A rollback leaves the file on disk with no row
Serving	getFileUrl() checks file_exists and is_readable, then returns BASE_URL + path	Direct URL - no authorisation check when the file is fetched

10. Frontend Reference (payment.php)

The view is a single file containing the stat cards, the accordion form, the DataTable, seven modals, a print-only section and roughly 1,800 lines of jQuery.

10.1 DataTable columns

#	Data	Sortable	Rendering
0	id	Yes	Bold, prefixed with #
1-3	requestee, beneficiary, client_name	Yes	HTML-escaped text
4	pay_for	Yes	Mapped to Import / Export / Local / Other / Pre Payment
5-7	payment_type, currency_short_name, expense_type_name	Yes	Escaped text
8	amount	Yes	Fixed to two decimals
9	mca_data	No	Eye button showing the reference count
10	computed	No	Status badge derived client-side from the five stage columns
11	created_at	Yes	toLocaleDateString()
12	computed	No	View, Print, conditional Edit and one conditional stage-action button

10.2 Modals

deptApprovalModal (adds the chargeback radio and amount), financeApprovalModal, mgmtApprovalModal, underProcessModal, paidApprovalModal (cash collector plus documents 3 and 4), viewModal (full record and the approval timeline), mcaRefModal (picker with live search), mcaRefsViewModal (reference table with copy-one and copy-all) and emailComposeModal. All four approval modals share buildApprovalDetailsHTML().

10.3 Print output

Printing does not go through mPDF. The handler fetches the record, builds two pages of HTML into #hiddenPrintSection, sets document.title to DEMANDE_DE_FONDS_{id} so the browser suggests that filename, then calls window.print(). A @media print block hides everything else. Page 1 carries the letterhead, the amount in French words via numberToFrenchWords(), the AUTORISATION grid and the RETRAIT signature boxes; page 2 lists the MCA references with a total. The footer still reads "Powered by TCPDF", which is a leftover from an earlier implementation.

10.4 Notable client-side helpers

Function	Role
escapeHtml()	textContent-based escaping applied to every interpolated value
parseExcelAmount()	Locale-tolerant numeric parser for imported amounts
formatAmountFrench()	Space thousands separator and comma decimal, for the printed form
numberToFrenchWords()	Amount in words including millions, thousands and centimes
validateMcaBeforeAdd() / validateMcaBatch()	Sequential existence + duplicate validation per reference
buildFileLinks()	Renders up to four document links with type-specific icons
ajaxRequest()	Wrapper that injects the CSRF token and routes failures to Toastr

Function	Role
loadStatusCounts()	Refreshes the eight cards; bound to the DataTable drawCallback

11. Security Analysis

The module already does several things well: every query is parameterised, the sort column is whitelisted, LIMIT values pass through `sprintf("%d")`, CSRF tokens are compared with `hash_equals()`, uploads are MIME-checked, and every value interpolated into the DOM goes through `escapeHtml()`. The findings below are the gaps that remain.

ID	Severity	Finding	Detail
S-01	CRITICAL	Missing ownership check in <code>store()</code> on update	<code>update()</code> accepts any <code>payment_id</code> . There is no verification that <code>created_by</code> equals the session user, nor that the record is still in an editable state. Any authenticated user can POST an id and overwrite another department's request - including one that is fully approved, and including the amount and the beneficiary. The UI hides the Edit button, but the endpoint does not.
S-02	HIGH	No authorisation in <code>get_single()</code>	The method calls <code>validateCSRF()</code> but never <code>validateSession()</code> and never applies <code>buildVisibilityConditions()</code> . Any id can be read, exposing amounts, motifs, approver identities, rejection notes and document paths across locations.
S-03	HIGH	CSRF is not enforced on GET endpoints	<code>validateCSRF()</code> returns immediately unless the method is POST. <code>get_list</code> is fetched by GET, and <code>delete/{id}</code> is a GET route that performs a destructive action. A crafted link could delete a request when opened by a management user.
S-04	HIGH	Paid stage does not verify Under Process for Bank payments	<code>validateApprovalWorkflow("paid_approve")</code> checks dept, finance and management but never <code>under_process</code> . Roles 11 and 48 hold both permissions, so a direct POST can mark a Bank payment as paid without the bank-processing step ever being recorded.
S-05	MEDIUM	Uploaded documents are publicly reachable	Files land under the web root and are served by direct URL. Anyone with the link - or anyone enumerating the predictable <code>time()_uniqid()_name</code> pattern - can fetch an invoice without being logged in. There is no authenticated download proxy.
S-06	MEDIUM	Unescaped session value in the view	<code>value="<?php echo \$_SESSION['user_data']['fullname']; ?>"</code> is printed into the requestee input without <code>htmlspecialchars()</code> , unlike every other echo on the page. A quote character in a stored name breaks the attribute and opens a stored-XSS path.
S-07	MEDIUM	<code>reject_request()</code> does not check stage ordering	A role may reject a stage that is not the current one - for example rejecting "dept" on a request that is already at the Paid stage - which overwrites the original approval record and its timestamp.
S-08	LOW	No location guard on <code>under_process_approval()</code>	Every other stage checks that the row belongs to the role's location. Under Process does not, so roles 11 and 48 act on all locations. This may be intentional (bank execution is centralised in Lubumbashi) but it is undocumented and inconsistent.
S-09	LOW	Server-side field validation is thinner than the client	The 10-500 character motif rule, the 200-character name limits and the "amount must be positive" rule exist only in HTML attributes and JavaScript. <code>store()</code> checks presence but not shape.
S-10	INFO	Verbose error_log on every request	The constructor logs directory existence and writability on each instantiation, and <code>uploadFile()</code> logs five lines per file. Exception messages are also returned to the client verbatim, which leaks paths in production.

12. Performance Analysis

ID	Severity	Finding	Detail
P-01	HIGH	check_mca_duplicate() scans and decodes the whole expense type	The query is <code>SELECT id, mca_data FROM payment_requests WHERE expense_type = ?</code> with no limit, followed by a <code>json_decode()</code> and a nested loop in PHP. On a table with tens of thousands of rows this is a full scan per reference - and the client calls it once per reference, so a 50-row Excel import triggers 50 full scans.
P-02	HIGH	Reference validation is sequential and chatty	<code>validateMcaBatch()</code> awaits each reference in turn and each one costs two round-trips (existence + duplicate). Fifty references is 100 sequential requests; on a slow DRC link the import can take minutes with the UI blocked behind a Toastr "please wait".
P-03	MEDIUM	Four queries per DataTable draw	<code>get_list()</code> runs the filtered count, an unfiltered visibility count and the page query; <code>drawCallback</code> then fires <code>get_status_counts()</code> for a fourth. Every sort click and page change repeats all four.
P-04	MEDIUM	Duplicate SELECT in the approval path	<code>update_approval()</code> loads the payment row, then calls <code>handleApproval()</code> which loads the identical row again and also re-runs <code>validateCSRF()</code> and <code>validateSession()</code> .
P-05	MEDIUM	Search across mca_data prevents index use	The global search adds <code>COALESCE(pr.mca_data, "") LIKE "%term%"</code> alongside ten other leading-wildcard LIKEs, forcing a full scan on every keystroke-driven search.
P-06	LOW	export_to_excel() selects pr.* for the entire visible set	No paging, no projection, no streaming - the whole result set is JSON-encoded into one response and parsed in the browser.
P-07	LOW	Constructor filesystem checks on every request	<code>file_exists</code> , <code>is_writable</code> and up to four <code>error_log</code> calls run before any routing decision, on every single AJAX hit including the ones that never touch a file.

13. Code Quality Findings

ID	Finding	Detail
Q-01	Permission matrix duplicated three times	Once in <code>update_approval()</code> , once in <code>reject_request()</code> and once in the DataTable action renderer. The copies have already drifted - the Under Process comment names role 34, which neither the condition nor the server allows.
Q-02	Location guards copy-pasted six times per method	Six near-identical if blocks in <code>update_approval()</code> and the same six again in <code>reject_request()</code> . A single role-to-location map would remove roughly 40 lines.
Q-03	Card predicates duplicated between counts and list	The eight status definitions are written twice, differing only by the <code>pr.</code> prefix. Any future change to a definition must be made in both places or the cards will disagree with the grid.
Q-04	delete() breaks the soft-delete convention	The rest of the ERP uses <code>display = "Y"</code> ; this method calls <code>deleteData()</code> and removes the audit trail permanently. It also has no UI caller.
Q-05	Hardcoded business data in the view	Three recipient addresses and one CC address for expense type 42, and the literal client id 68 that skips client validation. Both belong in configuration or a master table.
Q-06	Inconsistent response envelope	<code>getExpenseTypesByCategory()</code> and <code>get_list()</code> echo raw JSON while every other endpoint uses <code>jsonResponse()</code> . The JS wrapper has to special-case them.
Q-07	Leftover artefacts	The print footer credits TCPDF although the output is a browser print; a commented "email-payment" handler remains bound but no button renders it; the console banner still reads "Payment System v13.0".
Q-08	Client-side status logic diverges from the server	The badge renderer checks " <code>Bank + management = 1 + under_process IS NULL</code> " without also requiring dept and finance, so a partially approved Bank row can briefly display "Pending Under Process".
Q-09	Magic numbers throughout	<code>pay_for</code> values 0-4, expense types 25, 28 and 42, client 68 and <code>clearing_status</code> 7 all appear as bare literals with no named constant.

ID	Finding	Detail
Q-10	store() mutates state before commit	Old documents are unlinked and new ones moved into place before the transaction commits, so a rollback leaves the filesystem and the database out of step.

14. Recommended Fixes

The snippets below are drop-in for the current controller. They are ordered by the severity of the finding they close.

14.1 S-01 - ownership and editability guard in store()

Insert immediately after `$db->beginTransaction();` in the update branch, before any file is touched.

```
if ($action === "update" && $payment_id) {
    $ownSql = "SELECT created_by, dept_approval, finance_approval, management_approval,
                    under_process, paid_approval
                FROM payment_requests WHERE id = :id LIMIT 1";
    $own = $db->customQuery($ownSql, [':id' => $payment_id]);

    if (empty($own)) {
        $this->jsonResponse(false, 'Payment request not found');
    }

    $own = $own[0];

    if ((int)$own['created_by'] !== (int)$user['user_id']
        && (int)$user['role_id'] !== self::ROLE_MANAGEMENT) {
        $this->jsonResponse(false, 'You can only edit your own payment requests');
    }

    $isRejected = ($own['dept_approval'] == -1 || $own['finance_approval'] == -1
        || $own['management_approval'] == -1 || $own['under_process'] == -1
        || $own['paid_approval'] == -1);

    if (!$isRejected && $own['dept_approval'] !== null) {
        $this->jsonResponse(false, 'This request is already in the approval chain and can no longer be edited');
    }
}
```

14.2 S-02 - authorise get_single()

Add the session check and reuse the existing visibility helper so the endpoint cannot leak rows the user would never see in the grid.

```
public function get_single()
{
    $this->validateCSRF();
    $user = $this->validateSession();          // <-- was missing

    $id = $_POST['id'] ?? null;
    if (empty($id) || !is_numeric($id)) {
        $this->jsonResponse(false, "Invalid Payment ID");
    }

    $db = new Database();

    $visibility = $this->buildVisibilityConditions($user['role_id'], $user['user_id'], 'pr');
    $extraWhere = '';
    $params      = [':id' => $id];

    if (!empty($visibility['conditions'])) {
        $extraWhere = ' AND ' . implode(' AND ', $visibility['conditions']);
        $params      = array_merge($params, $visibility['params']);
    }

    // ... existing SELECT ...
    // WHERE pr.id = :id {$extraWhere}
    // LIMIT 1
}
```

14.3 S-04 - complete the Paid guard

Add to the paid_approve branch of `validateApprovalWorkflow()`.

```
case "paid_approve":
    // ... existing dept / finance / management checks ...
```

```

if ($payment['payment_type'] === 'Bank' && $payment['under_process'] != 1) {
    $this->jsonResponse(false, "Bank payments must be marked Under Process before they can be paid");
}
break;

```

14.4 S-03 - make delete() a POST with CSRF

```

public function delete($id = null)
{
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
        $this->jsonResponse(false, 'Invalid request method');
    }

    $this->validateCSRF();
    $user = $this->validateSession();

    if ($user['role_id'] != self::ROLE_MANAGEMENT) {
        $this->jsonResponse(false, "You don't have permission to delete payment requests");
    }

    $id = $id ?: ($_POST['id'] ?? null);
    if (!$id || !is_numeric($id)) {
        $this->jsonResponse(false, 'Invalid Payment ID');
    }

    $db = new Database();
    $db->updateData('payment_requests',
        ['display' => 'N', 'updated_at' => date('Y-m-d H:i:s'), 'updated_by' => $user['user_id']],
        ['id' => $id]);

    $this->jsonResponse(true, "Payment Request Deleted Successfully");
}

```

Adding display also requires appending `$whereConditions[] = "{$p}display = 'Y'";` at the top of `buildVisibilityConditions()` so soft-deleted rows disappear from the grid, the counters and the export at once.

14.5 P-01 - index the duplicate check

Two options. The lighter one keeps the JSON column and pushes the search into MySQL; the sturdier one normalises the references into a child table.

```

-- Option A: generated column + index (MySQL 5.7+), no schema migration of existing data
ALTER TABLE payment_requests
    ADD COLUMN mca_refs_flat TEXT
    GENERATED ALWAYS AS (JSON_UNQUOTE(JSON_EXTRACT(mca_data, '$[*].mca_ref'))) STORED;

CREATE INDEX idx_pr_expense_flat ON payment_requests (expense_type);

-- then in check_mca_duplicate():
-- SELECT id FROM payment_requests
-- WHERE expense_type = :expense_type
-- AND id != :payment_id
-- AND JSON_SEARCH(mca_data, 'one', :mca_ref, NULL, '$[*].mca_ref') IS NOT NULL
-- LIMIT 1

-- Option B (recommended): child table, makes references joinable and reportable
CREATE TABLE payment_request_refs_t (
    id INT AUTO_INCREMENT PRIMARY KEY,
    payment_id INT NOT NULL,
    mca_ref VARCHAR(100) NOT NULL,
    amount DECIMAL(15,2) NOT NULL DEFAULT 0,
    expense_type INT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    KEY idx_prr_payment (payment_id),
    KEY idx_prr_lookup (expense_type, mca_ref),
    CONSTRAINT fk_prr_payment FOREIGN KEY (payment_id)
        REFERENCES payment_requests(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- duplicate check becomes a single indexed lookup:
-- SELECT payment_id FROM payment_request_refs_t
-- WHERE expense_type = :expense_type AND mca_ref = :mca_ref AND payment_id != :payment_id

```

```
-- LIMIT 1
```

14.6 P-02 - batch the reference validation

One request for the whole batch instead of two per reference. Add this endpoint and have `validateMcaBatch()` call it once.

```
public function validate_mca_batch()
{
    $this->validateCSRF();
    $this->validateSession();

    $refs      = $_POST['refs']      ?? [];
    $payFor     = $_POST['pay_for']   ?? null;
    $clientId   = $_POST['client_id'] ?? null;
    $expenseType = (int)($_POST['expense_type'] ?? 0);
    $paymentId  = $_POST['payment_id'] ?? null;

    if (empty($refs) || !is_array($refs)) {
        $this->jsonResponse(false, 'No references supplied', []);
    }

    $refs = array_values(array_unique(array_filter(array_map('trim', $refs))));
    $refs = array_slice($refs, 0, 200);

    $db      = new Database();
    $result  = [];

    // ---- 1. existence in the tracking table (single IN query) ----
    $exists  = [];
    $mapping = [
        '0' => ['table' => 'imports_t', 'client_col' => 'subscriber_id', 'ref_col' => 'mca_ref'],
        '1' => ['table' => 'exports_t', 'client_col' => 'subscriber_id', 'ref_col' => 'mca_ref'],
        '2' => ['table' => 'locals_t', 'client_col' => 'client_id', 'ref_col' => 'mca_lt_reference'],
    ];

    if (isset($mapping[$payFor])) {
        $m      = $mapping[$payFor];
        $placeholders = [];
        $params  = [];

        foreach ($refs as $i => $r) {
            $placeholders[] = ":r{$i}";
            $params["r{$i}"] = $r;
        }

        $sql = "SELECT {$m['ref_col']} AS ref FROM {$m['table']}
            WHERE {$m['ref_col']} IN (" . implode(',', $placeholders) . ")";

        if ($clientId) {
            $sql .= " AND {$m['client_col']} = :client_id";
            $params[':client_id'] = $clientId;
        }

        if ($payFor === '0') {
            $sql .= " AND clearing_status != 7";
        }

        foreach ($db->customQuery($sql, $params) as $row) {
            $exists[$row['ref']] = true;
        }
    } else {
        foreach ($refs as $r) { $exists[$r] = true; } // pay_for 3 and 4 skip tracking
    }

    // ---- 2. duplicates (single indexed query against the child table) ----
    $dupes = [];
    if ($expenseType !== 25) {
        $placeholders = [];
        $params        = [':expense_type' => $expenseType];

        foreach ($refs as $i => $r) {
            $placeholders[] = ":d{$i}";
            $params["d{$i}"] = $r;
        }
    }
}
```

```

    }

    $sql = "SELECT mca_ref, payment_id FROM payment_request_refs_t
           WHERE expense_type = :expense_type
           AND mca_ref IN (" . implode(',', $placeholders) . ")";

    if ($paymentId) {
        $sql .= " AND payment_id != :payment_id";
        $params[':payment_id'] = $paymentId;
    }

    foreach ($db->customQuery($sql, $params) as $row) {
        $dups[$row['mca_ref']] = $row['payment_id'];
    }

    foreach ($refs as $r) {
        $result[] = [
            'mca_ref'    => $r,
            'valid'      => isset($exists[$r]) && !isset($dups[$r]),
            'exists'     => isset($exists[$r]),
            'duplicate' => isset($dups[$r]) ? $dups[$r] : null,
        ];
    }

    $this->jsonResponse(true, 'Validation complete', $result);
}

```

14.7 Q-01 and Q-02 - one permission table

Replace the three copies of the matrix and the twelve location if-blocks with a single map and two small helpers.

```

private static $STAGE_PERMISSIONS = [
    self::ROLE_DEPARTMENT          => ['dept'],
    self::ROLE_FINANCE_CASH        => ['finance'],
    self::ROLE_FINANCE_BANK        => ['finance'],
    self::ROLE_MANAGEMENT          => ['management'],
    self::ROLE_CASHIER             => ['paid'],
    self::ROLE_BANK_OFFICER        => ['under_process', 'paid'],
    self::ROLE_LOCATION_MANAGER    => ['dept', 'finance', 'management', 'paid'],
    self::ROLE_LOCATION_3_CASHIER  => ['paid'],
    self::ROLE_LOCATION_2_DEPT_MGMT => ['dept'],
    self::ROLE_LOCATION_2_MGMT     => ['management'],
    self::ROLE_LOCATION_2_FINANCE_PAID => ['finance', 'paid'],
    self::ROLE_LOCATION_1_MANAGER  => ['management'],
    self::ROLE_CASH_BANK_OFFICER   => ['finance'],
    self::ROLE_BANK_OFFICER_FULL_VIEW => ['under_process', 'paid'],
];

private static $ROLE_LOCATION = [
    self::ROLE_LOCATION_MANAGER      => 3,
    self::ROLE_LOCATION_3_CASHIER    => 3,
    self::ROLE_LOCATION_2_DEPT_MGMT  => 2,
    self::ROLE_LOCATION_2_MGMT       => 2,
    self::ROLE_LOCATION_2_FINANCE_PAID => 2,
    self::ROLE_LOCATION_1_MANAGER    => 1,
];

private function assertCanActOn($payment, $roleId, $stage)
{
    if (isset(self::$ROLE_LOCATION[$roleId])
        && (int)$payment['location_id'] !== self::$ROLE_LOCATION[$roleId]) {
        $this->jsonResponse(false, "You can only act on requests from Location "
            . self::$ROLE_LOCATION[$roleId]);
    }

    if (!isset(self::$STAGE_PERMISSIONS[$roleId])
        || !in_array($stage, self::$STAGE_PERMISSIONS[$roleId], true)) {
        $this->jsonResponse(false, "You don't have permission to perform this action");
    }

    if ($stage === 'paid'
        && $roleId === self::ROLE_LOCATION_2_FINANCE_PAID
        && $payment['payment_type'] === 'Bank') {

```

```

        $this->jsonResponse(false, "Bank payment execution is handled by Lubumbashi");
    }
}

```

The same array can be serialised into the page so the JavaScript stops hardcoding role numbers:

```

// in index()
$data['stage_permissions'] = self::$STAGE_PERMISSIONS[$_SESSION['user_data']['role_id']] ?? [];
$data['role_location']     = self::$ROLE_LOCATION[$_SESSION['user_data']['role_id']] ?? null;

// in the view
const MY_STAGES    = <?=json_encode($stage_permissions) ?>;
const MY_LOCATION  = <?=json_encode($role_location) ?>;

function canAct(stage, row) {
    if (!MY_STAGES.includes(stage)) return false;
    if (MY_LOCATION !== null && parseInt(row.location_id) !== MY_LOCATION) return false;
    return true;
}

```

14.8 P-04 - stop double-loading the payment row

Change `handleApproval()` to accept the row that `update_approval()` has already fetched.

```

// signature change
private function handleApproval($approvalType, $payment = null, $user = null)
{
    $user = $user ? : $this->validateSession();
    $id    = $_POST['id'] ?? null;

    if (!$id) {
        $this->jsonResponse(false, "Invalid Request");
    }

    if ($payment === null) {
        $db      = new Database();
        $result  = $db->customQuery("SELECT * FROM payment_requests WHERE id = :id", [':id' => $id]);
        $payment = $result[0] ?? null;
        if (!$payment) {
            $this->jsonResponse(false, "Payment request not found");
        }
    }

    // ... rest unchanged ...
}

// call site in update_approval()
$this->handleApproval($actionMap[$action], $payment, $user);

```

14.9 S-06 - escape the requestee value

```



```

14.10 P-03 - stop re-counting on every draw

Have `get_list()` return the counters inside the DataTables payload and drop the separate call from `drawCallback`.

```

// server: append to the echo in get_list()
echo json_encode([
    "draw"           => $draw,
    "recordsTotal"   => $recordsTotal,
    "recordsFiltered" => $recordsFiltered,
    "data"           => $data,
    "counts"         => $this->buildStatusCounts($userRole, $userId) // extracted from get_status_counts()
]);

// client: replace drawCallback
dataSrc: function (json) {
    if (json.counts) {

```

```
        $('#totalCount').text(json.counts.total);
        $('#waitingDeptCount').text(json.counts.waiting_dept);
        $('#waitingFinanceCount').text(json.counts.waiting_finance);
        $('#waitingMgmtCount').text(json.counts.waiting_mgmt);
        $('#waitingUnderProcessCount').text(json.counts.waiting_under_process || 0);
        $('#waitingPaymentCount').text(json.counts.waiting_payment);
        $('#paidCount').text(json.counts.paid);
        $('#rejectedCount').text(json.counts.rejected);
    }
    return json.data;
}
```


15. Suggested Database Indexes

Every card filter, the visibility clauses and the default ordering all hit the same handful of columns. These composites cover them.

```
-- visibility: location roles and creator lookups
CREATE INDEX idx_pr_location_created ON payment_requests (location_id, created_by);

-- visibility: bank officer and cashier paths
CREATE INDEX idx_pr_type_location ON payment_requests (payment_type, location_id);

-- card filters: the stage columns are always read together, in order
CREATE INDEX idx_pr_workflow ON payment_requests
    (dept_approval, finance_approval, management_approval, under_process, paid_approval);

-- default grid ordering and paging
CREATE INDEX idx_pr_created_at ON payment_requests (created_at DESC);

-- duplicate reference lookup (with the child table from 14.5 Option B)
CREATE INDEX idx_prr_lookup ON payment_request_refs_t (expense_type, mca_ref);

-- tracking-table lookups used by validate_mca_exists()
CREATE INDEX idx_imports_ref_client ON imports_t (subscriber_id, mca_ref);
CREATE INDEX idx_exports_ref_client ON exports_t (subscriber_id, mca_ref);
CREATE INDEX idx_locals_ref_client ON locals_t (client_id, mca_ref, reference);
```

Caveat. The workflow composite only helps when the leading columns are constrained. The PAID and REJECTED cards, which filter on the trailing columns, will still scan - if those two views become slow, add narrow single-column indexes on `paid_approval` and on the rejection predicate instead.

16. Appendix - Endpoint Index

Method	Route	Auth	CSRF	Visibility filter
index	GET payment/	session (implicit)	n/a	n/a
get_status_counts	POST payment/get_status_counts	yes	yes	yes
get_list	GET payment/get_list	yes	no (GET)	yes
store	POST payment/store	yes	yes	no - see S-01
get_single	POST payment/get_single	no - see S-02	yes	no - see S-02
update_approval	POST payment/update_approval	yes	yes	role + location guard
under_process_approval	POST payment/under_process_approval	yes	yes	role only - see S-08
reject_request	POST payment/reject_request	yes	yes	role + location guard
validate_mca_exists	POST payment/validate_mca_exists	no	yes	n/a
check_mca_duplicate	POST payment/check_mca_duplicate	no	yes	n/a
get_mca_refs_by_client	POST payment/get_mca_refs_by_client	no	yes	client scoped
getExpenseTypesByCategory	POST payment/getExpenseTypesByCategory	no	yes	n/a
export_to_excel	POST payment/export_to_excel	yes	yes	yes
delete	GET payment/delete/{id}	yes (role 1)	no (GET)	n/a

Priority order for remediation

Order	Item	Effort
1	S-01 ownership guard in store()	Small - one block
2	S-04 Under Process check before Paid	Trivial - four lines
3	S-02 authorise get_single()	Small - reuse the visibility helper
4	S-03 convert delete() to POST + soft delete	Small
5	S-06 escape the requestee value	Trivial
6	P-01 / P-02 child table + batch validation endpoint	Medium - migration plus a backfill script
7	Q-01 / Q-02 single permission map, exposed to JS	Medium - touches three methods and the view
8	P-03 / P-04 fold the counters into get_list, pass the row through	Small
9	S-05 authenticated download proxy for documents	Medium
10	Q-03 extract the card predicates into one private method	Small

This document was produced by static reading of `PaymentController.php` and `views/payment/payment.php` as supplied. Column types, the routing convention and the behaviour of the Database wrapper are inferred from usage and from the wider ERP conventions; verify them against the live schema before applying any migration.